Algoritmos e Programação: Orientação a Objetos

Revisão de programação em C

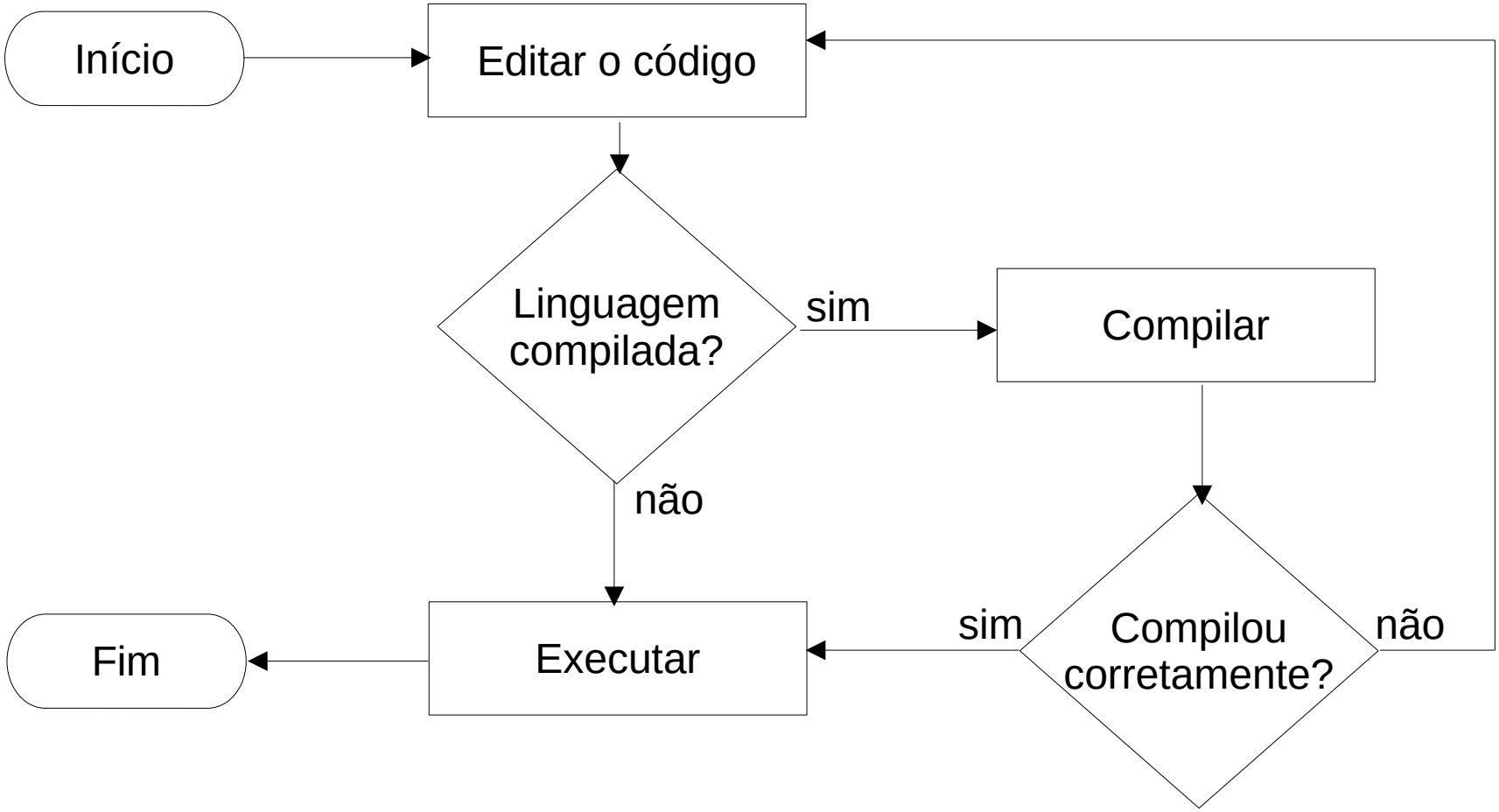
Profa. Andriele Busatto do Carmo

acarmo@unisinos.br

UNISINOS

Programação

- Fornecemos instruções para resolver problemas.

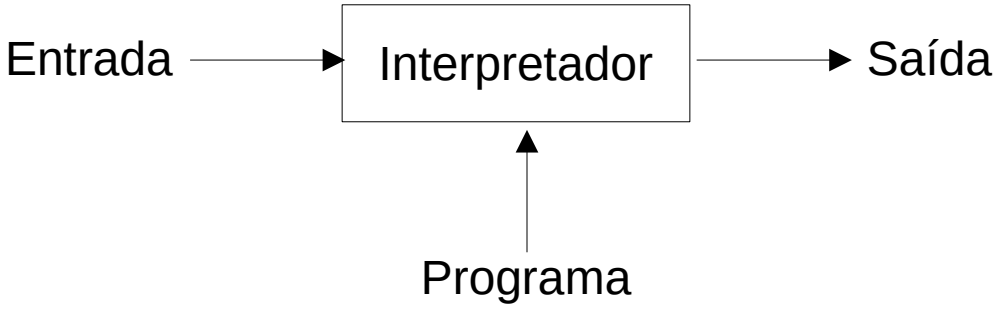


Definições básicas

- Dois tipos de tradutores:

  1. Interpretadores

  - Instruções definidas na linguagem de alto nível são executadas diretamente.
  - Traduzem o código fonte em código alvo (ou linguagem de máquina) a cada linha.
  - Executam o que foi traduzido sem gerar arquivos a partir da tradução.

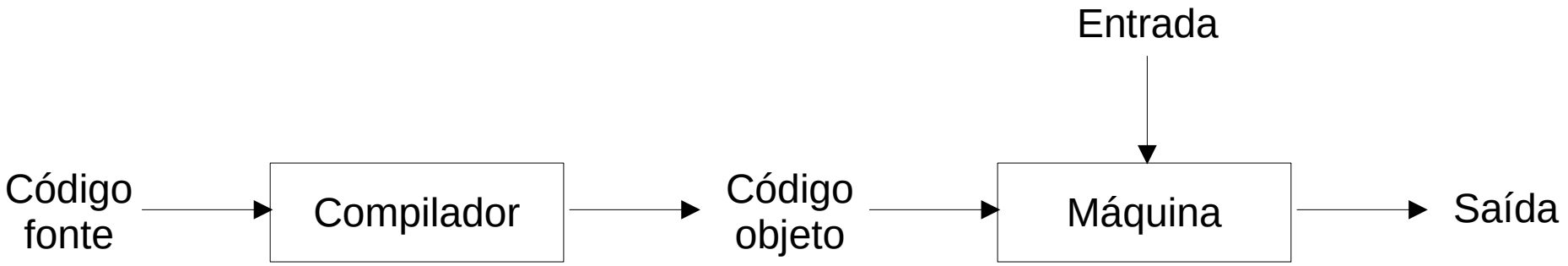


Definições básicas

- Dois tipos de tradutores *(continua...)*:

  2. Compiladores

  - Traduzem um programa escrito em uma linguagem fonte para um programa equivalente em linguagem de máquina.
    - São chamados **programa fonte** (ou código fonte) e **programa objeto** (ou código objeto).



Gerando código executável

- **Compilação**
  - Outros passos:

    1. O **pré-processador** mapeia instruções escritas em uma linguagem de alto nível estendida para instruções da linguagem de programação original.

       - Exemplos: processamento de macros; inclusão de arquivos; substituição de código não oferecido pelo compilador, mas suportado por ele; extensão da linguagem.

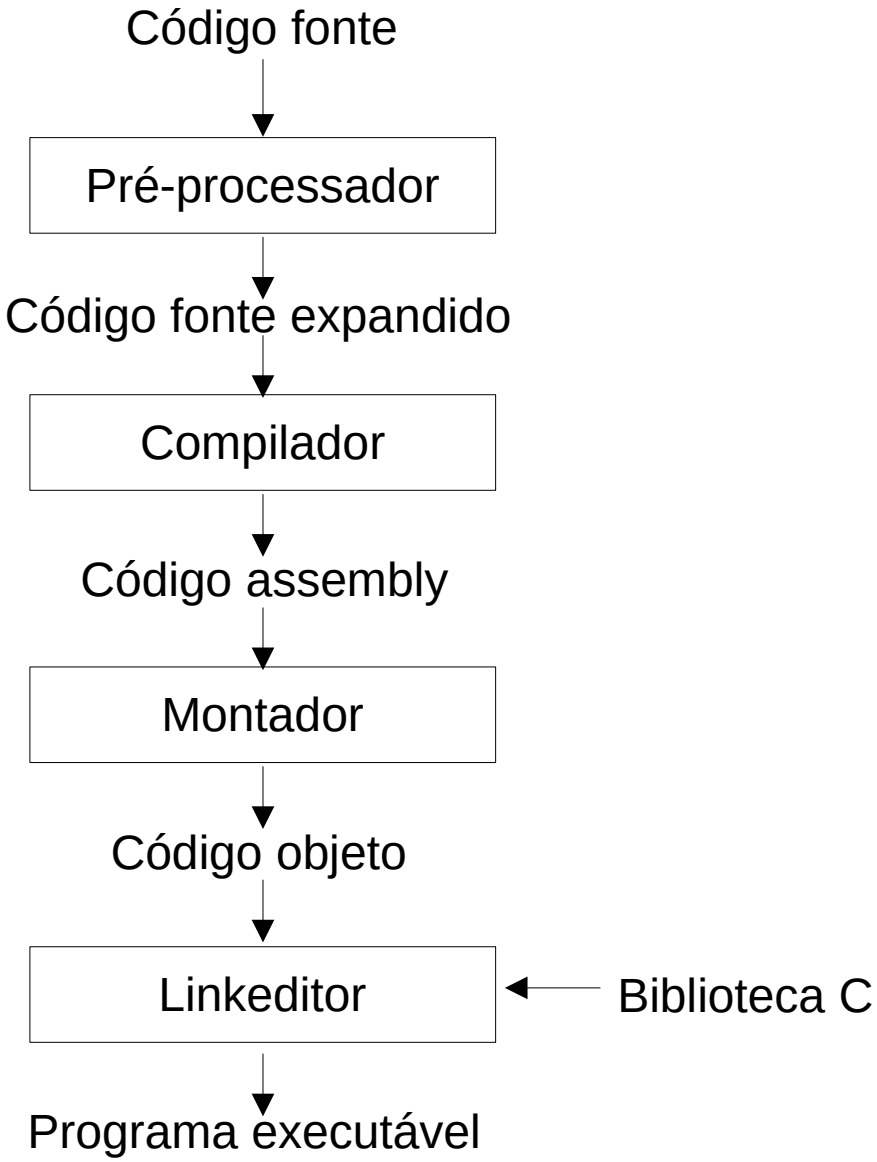
    2. O **compilador** analisa o código-fonte e converte para linguagem assembly (versão mnemônica da linguagem de máquina).

    3. O **montador** traduz o código assembly para código objeto, uma versão em linguagem de máquina do programa fonte. Porém, esta é uma forma intermediária, que não pode ser lida pelo programador e nem pela máquina. Esta forma intermediária existe porque todos os programas em C devem ser linkados com rotinas de suporte da biblioteca de execução C.

    4. O **linkeditor** junta o código objeto com as bibliotecas necessárias para gerar o programa executável.

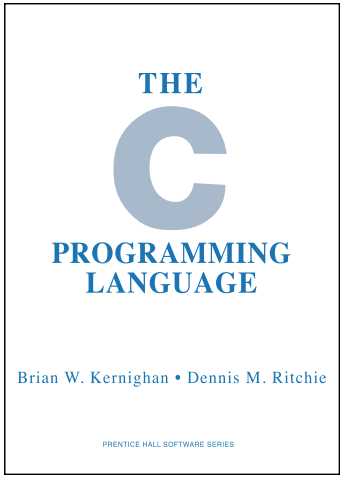
Gerando código executável

- **Compilação**



Linguagem C

- Histórico
  - Criada na década de 70
  - Por Dennis Ritchie nos laboratórios Bell
  - Lançada com o Sistema Operacional UNIX
  - Em 1985 foi estabelecido o padrão C ANSI
    - Pelo *American National Standards Institute* (ANSI)



Linguagem C

- Principais características
  - Linguagem compilada
  - Portável
  - Recursos de mais “baixo nível”
  - Geração de código eficiente
  - Simplicidade
  - Possibilidade de uso para propósitos variados
    - Desenvolvimento de software básico (Sistemas Operacionais, compiladores, etc).

Compilador GCC (*GNU Compiler Collection*)

- Compilador que utilizaremos para os códigos escritos na linguagem C.

- Utilizaremos o padrão C ANSI.

Estrutura básica de um programa em C

```
// Estrutura de um programa em C


#include <arquivo_cabecalho.h>


void main() {


    declaração de variáveis


    instrução 1;
    instrução 2;
    …
    instrução n;
}
```

```
// Exemplo de um programa em C


void main() {
    int a;              //declara a variável a
    a = 4 + 5;          //soma 4 com 5
}
```

Estrutura básica de um programa em C

- Estruturado em funções.
- O programa possui uma ou mais funções.
- A função principal, que inicia a execução do programa e chama todas as demais funções, é chamada **main**.
- Todas as funções, exceto a **main**, podem ter o nome que o programador desejar (desde que siga as regras de nomeação).
- Todo programa em C inicia sua execução chamando a função **main()**. **Por isso, a sua declaração é obrigatória**.

Identificadores

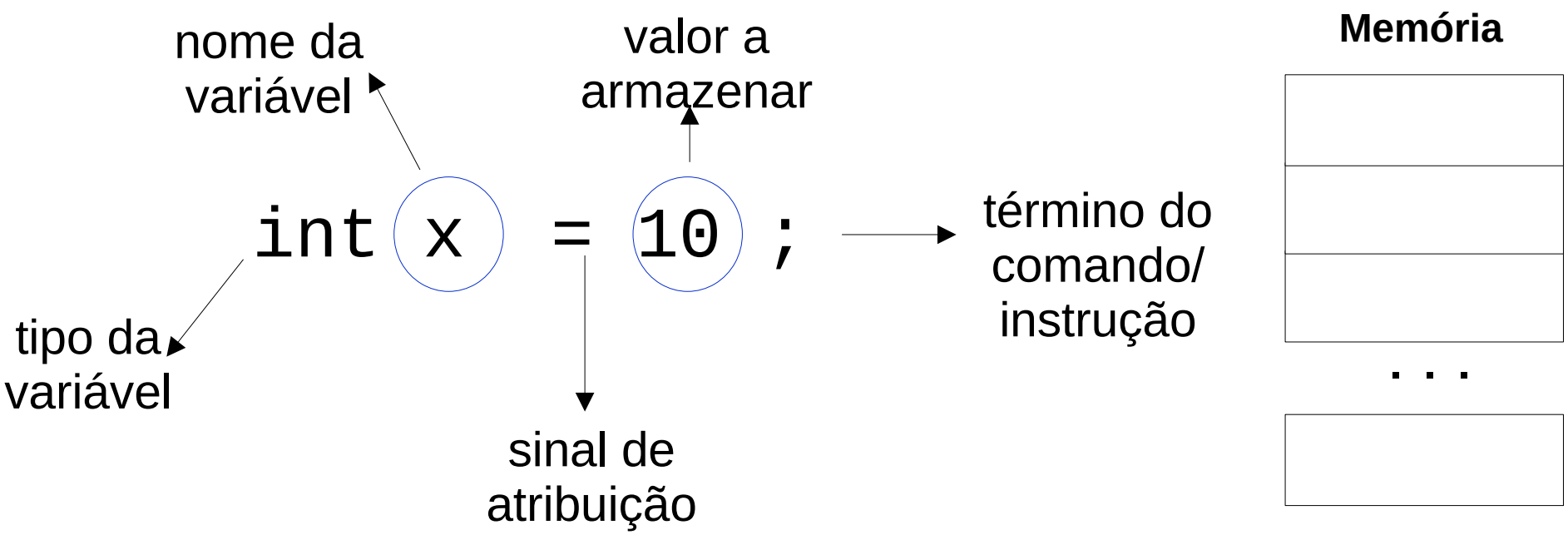
- Nomes de podem ser dados a variáveis e funções.

- Há regras para a escolha dos nomes:
  - deve iniciar por uma letra ou por um "_" (*underscore*);
  - a partir do segundo caracter pode conter letras, números e *underscore*;
  - C é uma linguagem *case-sensitive* (letras minúsculas e maiúsculas diferem identificadores). Por isso, **`aula`** e **`Aula`** são identificadores diferentes;
  - deve-se usar nomes significativos dentro do contexto do programa;
  - costuma-se usar maiúsculas e minúsculas para separar palavras: "aulaDeProgramacao";
  - deve ser diferente dos comandos da linguagem;
  - Exemplos: `aula_2`, `aula2`, `_nome`, Nome, `idade`.

Variáveis

- Posição de memória que identificada por um nome.

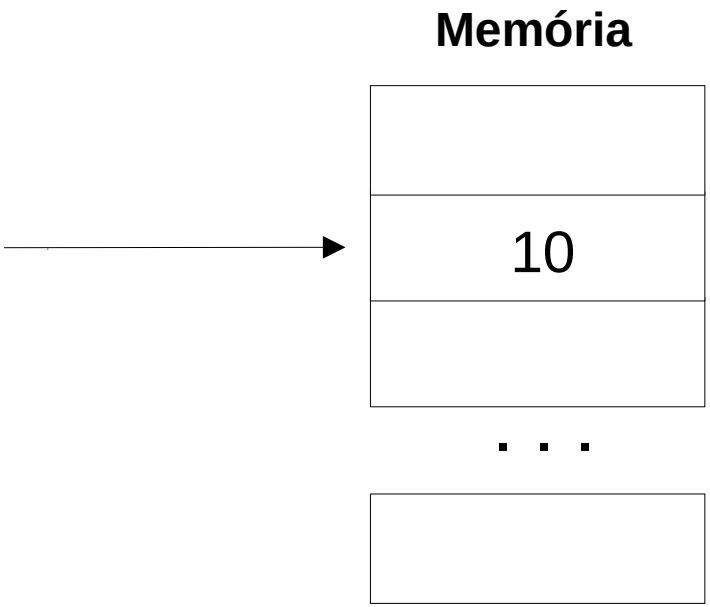
- Exemplo: como armazenar o número 10 na memória?



Variáveis

- No exemplo anterior:

```
int x    =    10 ;
```



Posição de memória chamada **x** é criada e nela é inserido o valor 10.

Tipos de variáveis

- Variáveis em C tem **tipo**.
- O **tipo** define os valores que uma variável pode armazenar.
- Cada **tipo** ocupa uma determinada quantidade de memória.

Tipos de variáveis

Tipo	Em C	Valores válidos
Caractere	char	Letras e símbolos: ‘a’, ‘B’, ‘*’, ‘#’, ‘2’
Inteiro	int	Valores inteiros: 1, -45, 89, -1004
Real	float	Valores reais: 1.56, -6.986, -56.98
Real	double	Valores reais com maior precisão do que float

Declaração de variáveis

- Toda variável em C **deve ser declarada antes** de ser usada.

- Podem ou não ser inicializadas no momento de declaração.

- Exemplos:

```
int var, qtdeDias = 15;

float x = 10.5;

double numero = 35.6;

char a, c = '@';
```

Variáveis de mesmo tipo podem ser declaradas juntas, separadas por uma vírgula.

Comando de saída: `printf`

- Função que exibe um ou mais dados na tela.

- São necessários dois parâmetros:

  1. uma string de formato que define, por meio de caracteres especiais, os tipos dos dados a serem impressos e suas posições na linha de impressão.

  2. um dado a ser exibido/impresso na tela. Pode ser de qualquer um dos tipos vistos anteriormente.

Observação: String é uma sequência de caracteres. Exemplo: “Aula de Programação!”

Comando de saída: `printf`

- Sintaxe:

```
printf(string_formato, dado_a_exibir);
```

OU, outra maneira de entender:

```
printf(string_controle, lista_argumentos);
```

- Exemplo:

Dado que será impresso na tela.

```
printf("%d", 35);
```

Será substituído pelo dado que vem após a vírgula.

Comando de saída: `printf`

- Mais exemplos:

```
printf(“Esta é uma mensagem que tem o caracter: %c”, ‘a’);

printf(“%s %s”, “primeira string”, “segunda string”);

printf(“Hello World!”);
```

Comando de saída: `printf`

- Para utilizar o comando, é necessário incluir o cabeçalho da biblioteca **stdio** no código do programa:

```
#include <stdio.h>
```

- Exemplo:

```
#include <stdio.h>

void main() {
   printf(“Hello World!”);
}
```

Comando de saída: `printf`

- Cada tipo de dado requer o uso de um identificador específico para sua impressão:

Código	Tipo
`%c`	`char`
`%d`	`int`
`%f`	`float`
`%lf`	`double`
`%s`	String
`%%`	Imprime %

Operadores aritméticos básicos

Em C	Significado
+	Soma
-	Subtração
*	Multiplicação
/	Divisão

Exercícios

Exercícios

1. Escreva um programa que imprime o nome da sua cidade natal.

2. Escreva um programa que imprime:

- na primeira linha o seu nome
- na segunda linha a sua idade
- na terceira linha a sua altura

3. Escreva um programa que imprime o número 75.897756 com 1, 2 e 5 casas decimais.

Exercícios

4. Escreva um programa que imprime:

- o resultado da soma
- o resultado da multiplicação (produto)
- o resultado da subtração
- o resultado da divisão

- Considerando dois valores de tipo inteiro armazenados em duas variáveis diferentes e definidos por você.